



河南大學
HENAN UNIVERSITY

计算机组成原理

主讲人：胡 萍

明德，新民

止于至善





4.3 高速缓冲存储器

□ 4.3.1 概述

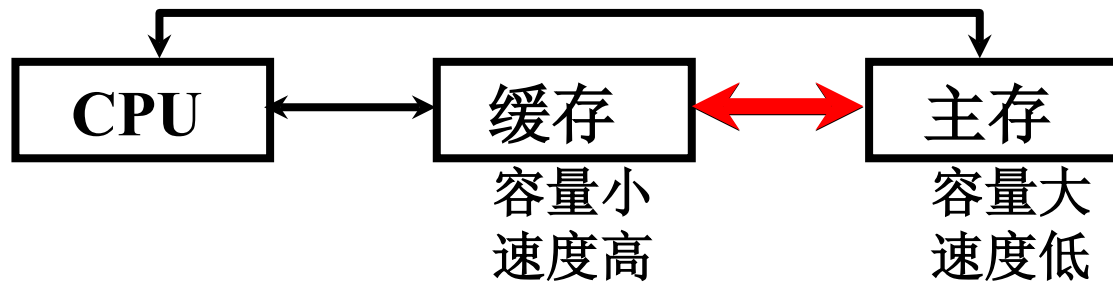
□ 4.3.2 Cache—主存地址映射

□ 4.3.3 替换策略

4.3.1 概述

1. 问题的提出

- 避免 CPU “空等” 现象
- CPU 和主存（DRAM）的速度差异



根据程序访问的局部性原理，只要将CPU近期要用到的程序、数据或访问频率比较高的数据（热数据）提前从主存调入Cache中，就可以做到CPU在一定时间内只访问Cache。

程序局部性：程序在执行时呈现出局部性规律，即在一段时间内整个程序的执行仅限于程序中的某一部分，而执行程序所需的指令和数据也仅局限于某个存储区域。



补充：程序局部性

```
int i, sum=0;  
for (i = 0; i < 100; i++)  
    sum += a[i];  
return sum;
```

□ **空间局部性**：内存某区域刚被访问，其相邻区域也即将被访问。主要体现在：

- 数据访问：数组元素、结构体、数据库记录等；
- 指令访问：顺序访问的指令。

□ **时间局部性**：内存某区域刚被访问，该区域在未来可能会被多次访问。主要体现在：

- 数据访问：局部变量，计数器，指针等被重复使用；
- 指令访问：重复使用的循环体、子函数。



举例：哪个程序具有更好的局部性？

假设M、N均为4，按字节编址，每个数组元素占4字节，数据在主存中的存放如下：

100H	a[0][0]
104H	a[0][1]
108H	a[0][2]
10CH	a[0][3]
110H	a[1][0]
114H	a[1][1]
.....	

空间局部性更好

```
int a[M][N];  
for (i = 0; i < M; i++)  
    for (j = 0; j < N; j++)  
        sum += a[i][j];
```

```
int a[M][N];  
for (j = 0; j < N; j++)  
    for (i = 0; i < M; i++)  
        sum += a[i][j];
```

1. 主存和CPU之间增加高速缓冲存储器的目的是()。

A

A

解决CPU和主存之间的速度匹配问题

B

扩大主存容量

C

既扩大主存容量又提高存取速度

D

以上答案都不对

提交



2. 【2017 程序局部性】某C语言程序段如下：

```
for (i=0; i<=9; i++){
    tem = 1;
    for (j=0; j<i; j++) temp * =a[j];
    sum+=temp;
}
```

下列关于数组a的访问局部性的描述中，正确的是()。

- ☒ A 时间局部性和空间局部性皆有
- ☐ B 无时间局部性，有空间局部性
- ☐ C 有时间局部性、无空间局部性
- ☐ D 时间局部性和空间局部性皆无

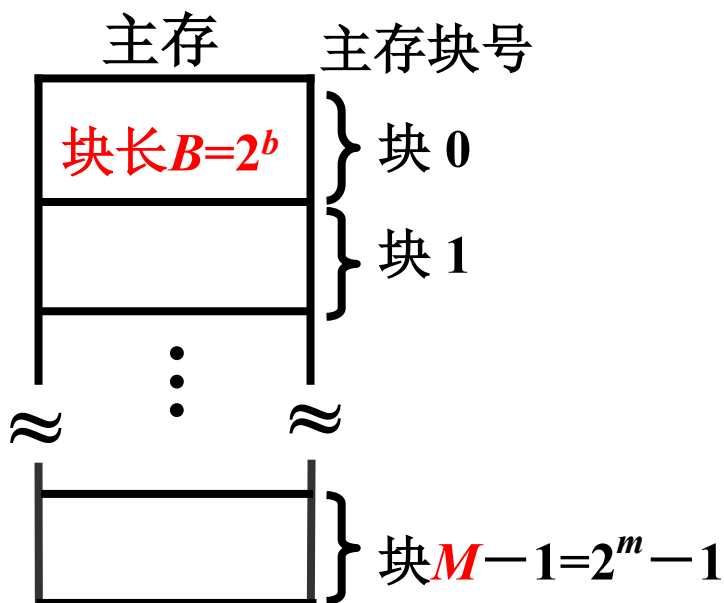
A

提交

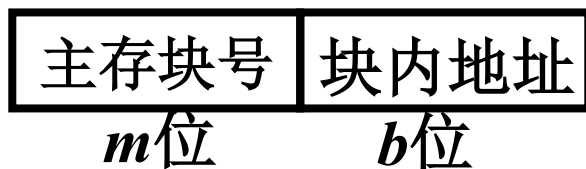


2. Cache 的工作原理

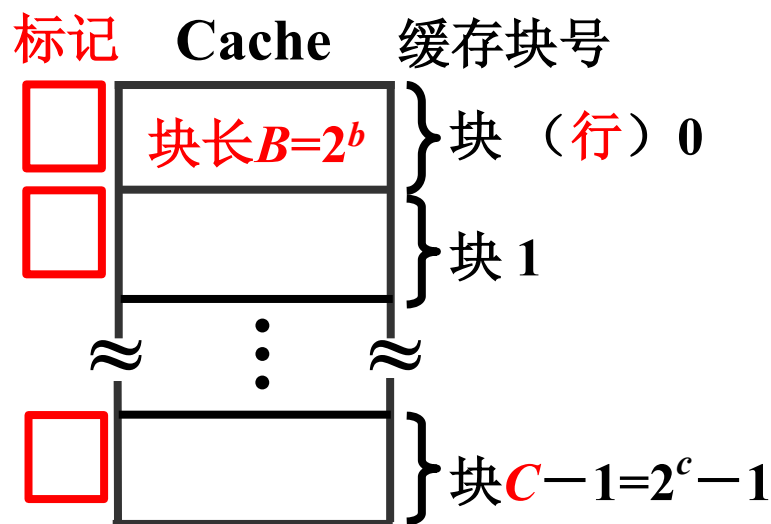
(1) 主存和缓存的编址



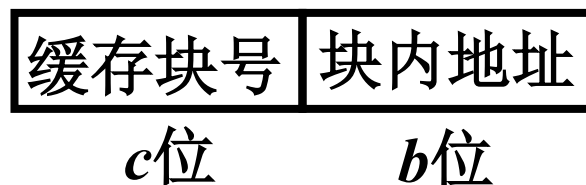
主存地址:



主存和缓存按块交互
块的大小（块长 $B=2^b$ ）相同



缓存地址:





(2) 命中与未命中

主存共有 M 块，缓存共有 C 块，且 $M \gg C$ 。

命中：

主存块已调入 缓存

主存块与缓存块 建立 了对应关系

用 **标记**记录 与某缓存块建立了对应关系的 主存块号

未命中：

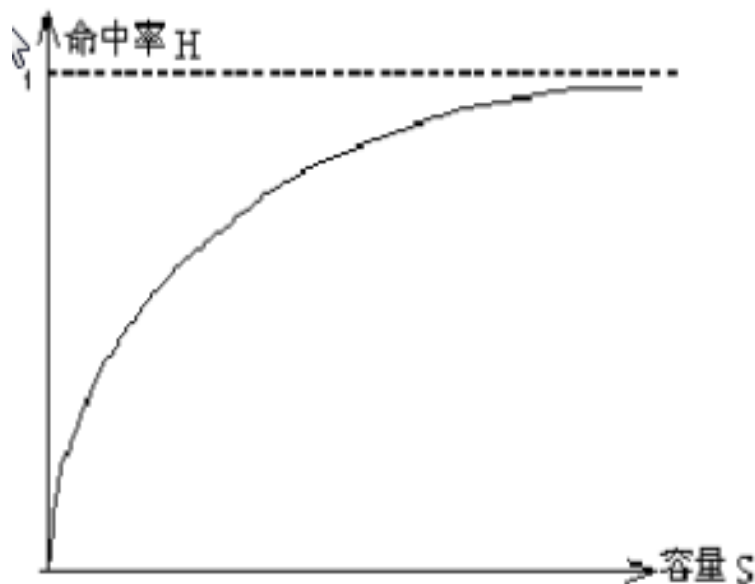
主存块 未调入 缓存

主存块与缓存块 未建立 对应关系



(3) Cache 的命中率

命中率：CPU 欲访问的信息在 Cache 中的 比率



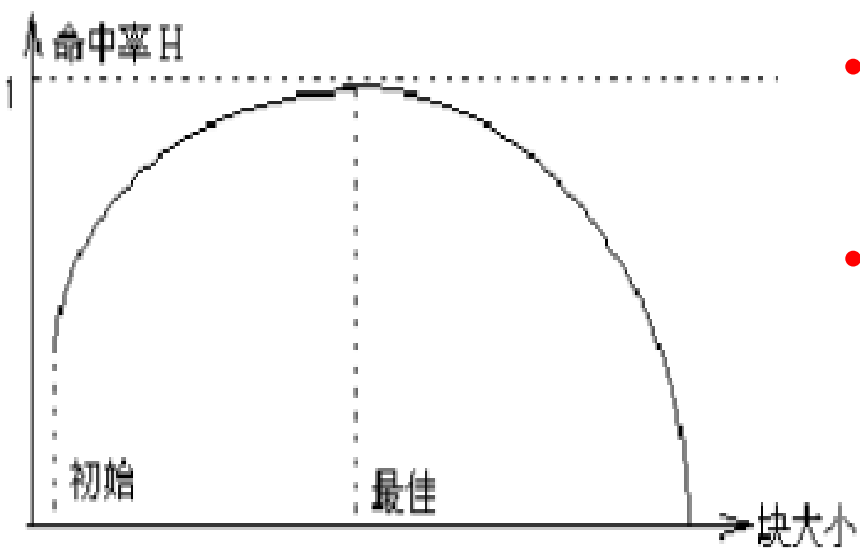
Cache容量越大，命中率越高

命中率 与 Cache 的 **容量** 的关系



(3) Cache 的命中率

块长：一般每块可取 4 ~ 8 个字或取一个存取周期内从主存调出的信息长度。



命中率与 Cache 的
块长的关系

- 增大块长可将更多有用字存入缓存，可提高命中率。
- 若块长过长，使新装入的有用数据反而少于替换掉的有用数据，会使缓存块数减少，导致少数块刚装入就被覆盖。反而降低命中率。



(3) Cache 的命中率

设 N_c 为访问Cache的总命中次数， N_m 为访问主存的总次数，
则命中率 h 为

$$h = \frac{N_c}{N_c + N_m}$$



● (4) Cache –主存系统的效率

设 t_c 为命中时的Cache访问时间， t_m 为未命中时的主存访问时间， $1-h$ 表示未命中率，则**Cache-主存系统的平均访问时间 t_a** 为：

$$t_a = ht_c + (1-h)t_m$$

此题未设置答案，请点击右侧设置按钮

3. Cache用 [标记] 记录与某缓存块建立了对应关系的主存块号；Cache的命中率与Cache的 [容量] 与 [块长] 有关。

正常使用填空题需3.0以上版本雨课堂

作答



4. 【2009 命中率】假设某计算机的存储系统由Cache和主存组成，某程序执行过程中访存1000次，其中访问Cache缺失（未命中）50次，则Cache 的命中率是()。

D

A

5%

B

9.5%

C

50%

D

95%

提交





5. 假设CPU执行某段程序时,其命中率为0.97, Cache的存取周期为50ns, 主存的存取周期为200ns, 则系统的平均访问时间为()。

C

A

53.5ns

B

50ns

C

54.5ns

D

55.5ns

提交





(4) Cache –主存系统的效率

Cache –主存系统的效率 e 与命中率有关

$$e = \frac{\text{访问 Cache 的时间}}{\text{平均访问时间}} \times 100\%$$

设 Cache 命中率为 h ，访问 Cache 的时间为 t_c ，访问主存的时间为 t_m ，平均访问时间为 t_a

$$\text{则 } e = \frac{t_c}{t_a} = \frac{t_c}{ht_c + (1-h)t_m} \times 100\%$$



【例4.7】 假设CPU执行某段程序时，共访问Cache命中2000次，访问主存50次。已知Cache的存取周期为50ns，主存的存取周期为200ns。求Cache-主存系统的命中率、效率和平均访问时间。

解：

(1) Cache命中率为 $2000 / (2000 + 50) = 0.97$

(2) 访问效率 $e = \frac{\text{访问 Cache 的时间}}{\text{平均访问时间}} \times 100\%$

$$= \{50 / [0.97 \times 50 + (1 - 0.97) \times 200]\} \times 100\%$$

$$= 91.7\%$$

(3) 平均访问时间为：

$$0.97 \times 50 + (1 - 0.97) \times 200 = 54.5\text{ns}$$



【例4.10】 假设Cache的工作速度是主存的5倍，且Cache被访问命中的概率为95%，则采用Cache后，存储器的性能提高多少？

解： 设Cache的存取周期为 t ，主存的存取周期为 $5t$ ，则系统的平均访问时间为：

$$t_a = ht_c + (1-h)t_m = 0.95t + 0.05 \times 5t = 1.2t$$

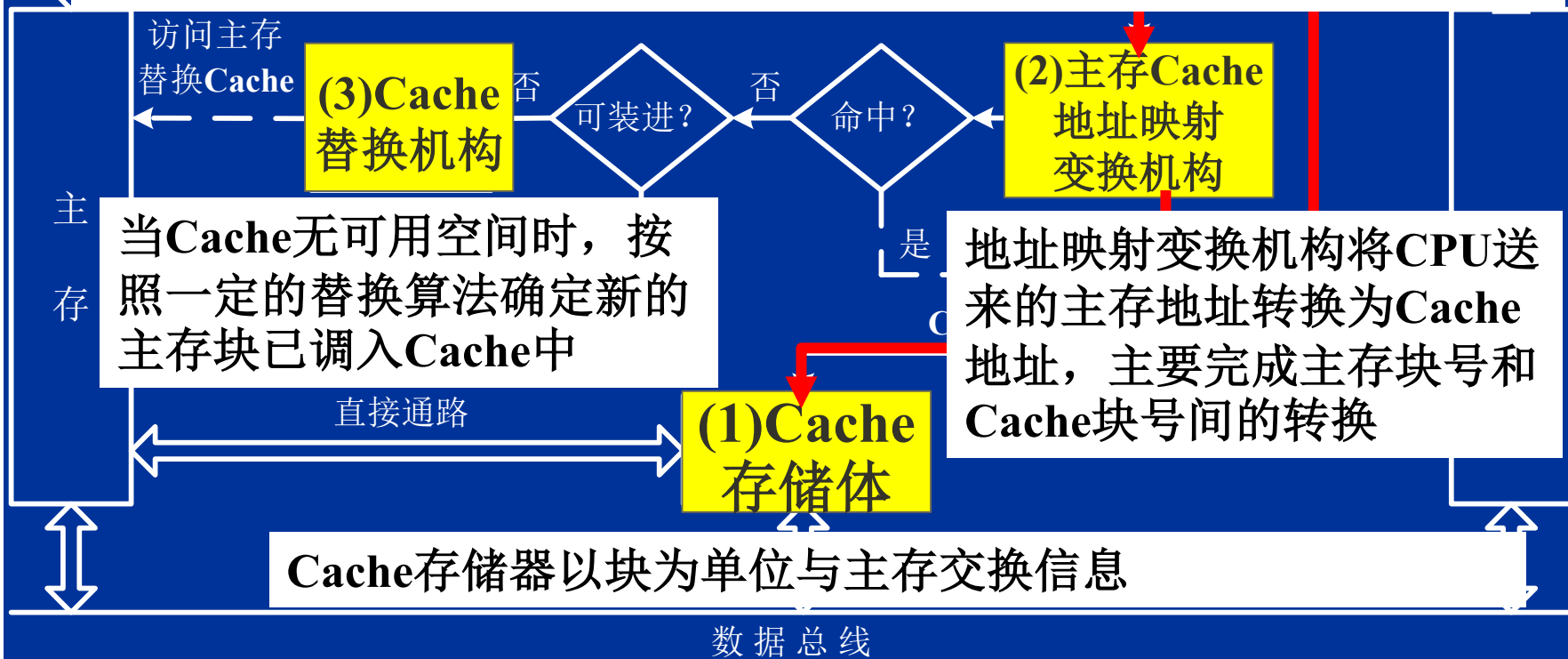
性能提高了 $5t \div 1.2t - 1 = 3.17$ 倍



3. Cache 的基本结构

地址总线

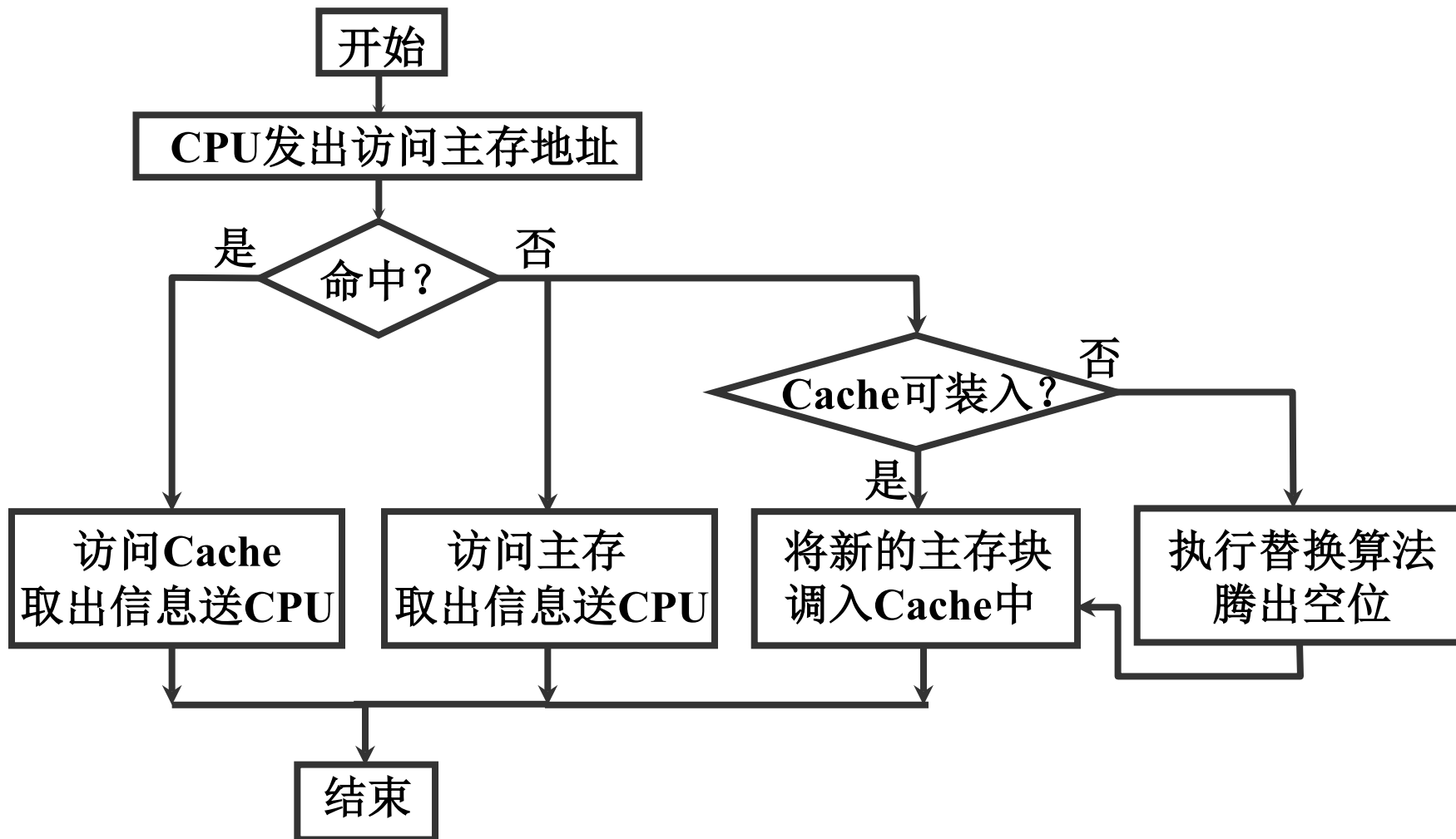
Cache对用户透明，用户编程时所用地址为主存地址，即将主存块调入Cache 的任务全由CPU（硬件）完成。





(4) Cache的读写操作

Cache读操作流程



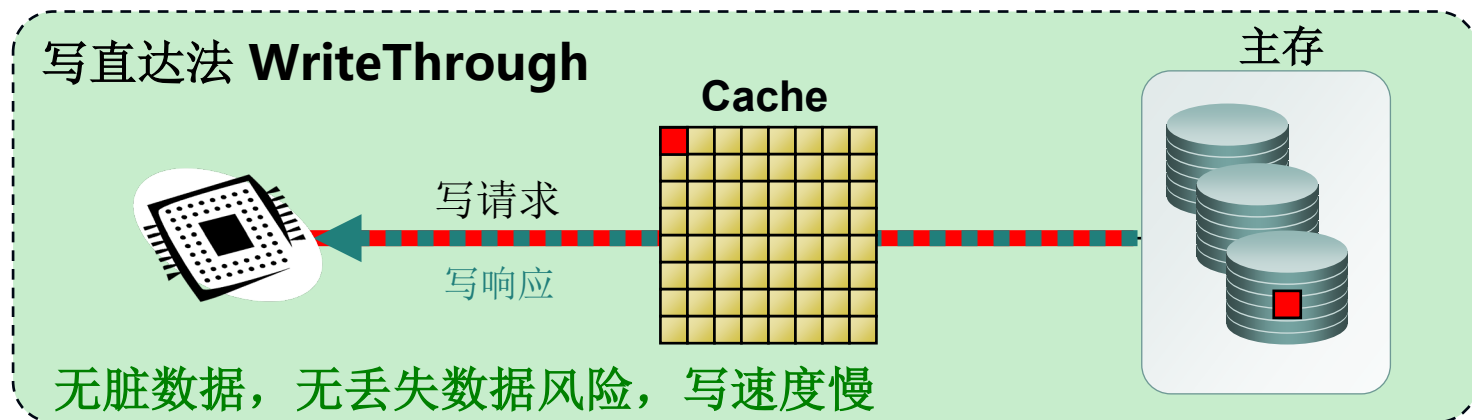


● (4) Cache的读写操作

写操作要保证Cache 和主存信息一致性

□ 写直达法 (Write-through)

- 写操作时数据既写入Cache又写入主存，写时间就是访问主存的时间，写速度慢。
- 读操作时不涉及对主存的写操作，更新策略比较容易实现。



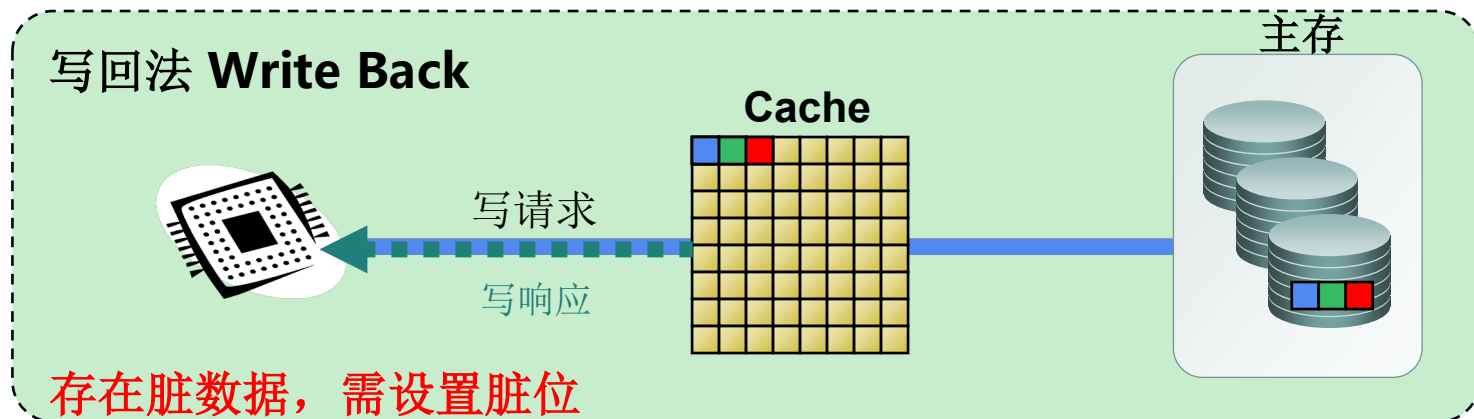


(4) Cache的读写操作

写操作要保证Cache 和主存信息一致性

□ 写回法 (Write – back)

- 写操作时只把数据写入 Cache 。写时间就是访问 Cache 的时间，速度快。
- 当发生改写的 Cache 数据被替换出去时，需写回主存，因此需**脏位**标志位。脏位为1表示发生改写。
- 读Cache 未命中时，若被替换的Cache 数据块发生改写，需写回主存，此时涉及主存写操作，增加了Cache 复杂性。



7. Cache的写操作比较复杂，需要保持Cache与主存内容一致。目前主要采用的方法有写直达法和写回法。速度较快的是 **写回法**，因为其写操作时间就是访问Cache的时间，但是在读Cache失效且需要发生数据块替换时，需要将被替换的块一次写回主存，增加了Cache的复杂性。比较容易实现的是 **写直达法**，其速度较慢，因为其写操作时间就是访问主存的时间。



8. 写直达法的特点是（ ），写回法的特点是（ ）。

C

A

写操作时间就是访问主存的时间，读操作时不涉及对主存的写操作

B

写操作时间就是访问Cache的时间，读操作时不涉及对主存的写操作

C

写操作时间就是访问主存的时间，读操作时涉及对主存的写操作

提交





4. Cache 的改进

采用多个Cache的含义：

- 增加Cache的级数
- 将统一的Cache变成分立的Cache



4. Cache 的改进

(1) 单一缓存和两级缓存

- 单一缓存：CPU和主存之间只设一个缓存
- 两级缓存：片内缓存和片外缓存

(2) 统一缓存和分立缓存

- 统一缓存：指令和数据都存放在同一缓存内
- 分立缓存：指令缓存和数据缓存
 - 与主存结构和指令的控制方式有关；
 - 超前控制或流水线控制，一般采用分立缓存



Cache关键技术

由Cache的读写流程，可知实现Cache时需解决以下关键技术：

❑ 数据查找 **Data Identification**

如何快速判断数据是否在Cache中

❑ 地址映射 **Address Mapping**

主存块如何放置到Cache中，如何将主存地址转换为Cache地址

❑ 替换策略 **Placement Policy**

Cache中无可可用空间后，采用什么策略对Cache块进行淘汰或替换

❑ 写入策略 **Write Policy**

如何既保证Cache块与主存块的数据一致性，又尽量提升效率



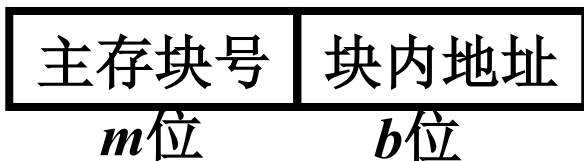
4.3 高速缓冲存储器

□ 4.3.1 概述

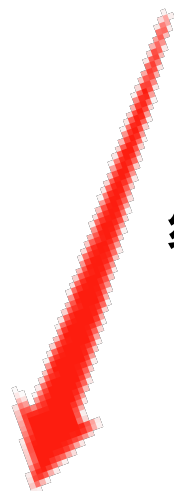
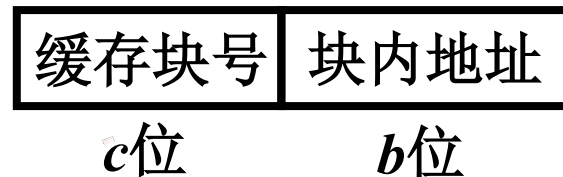
□ 4.3.2 Cache—主存地址映射

□ 4.3.3 替换策略

主存地址：



缓存地址：



主存地址 → Cache 地址 即 主存块号 → Cache块号

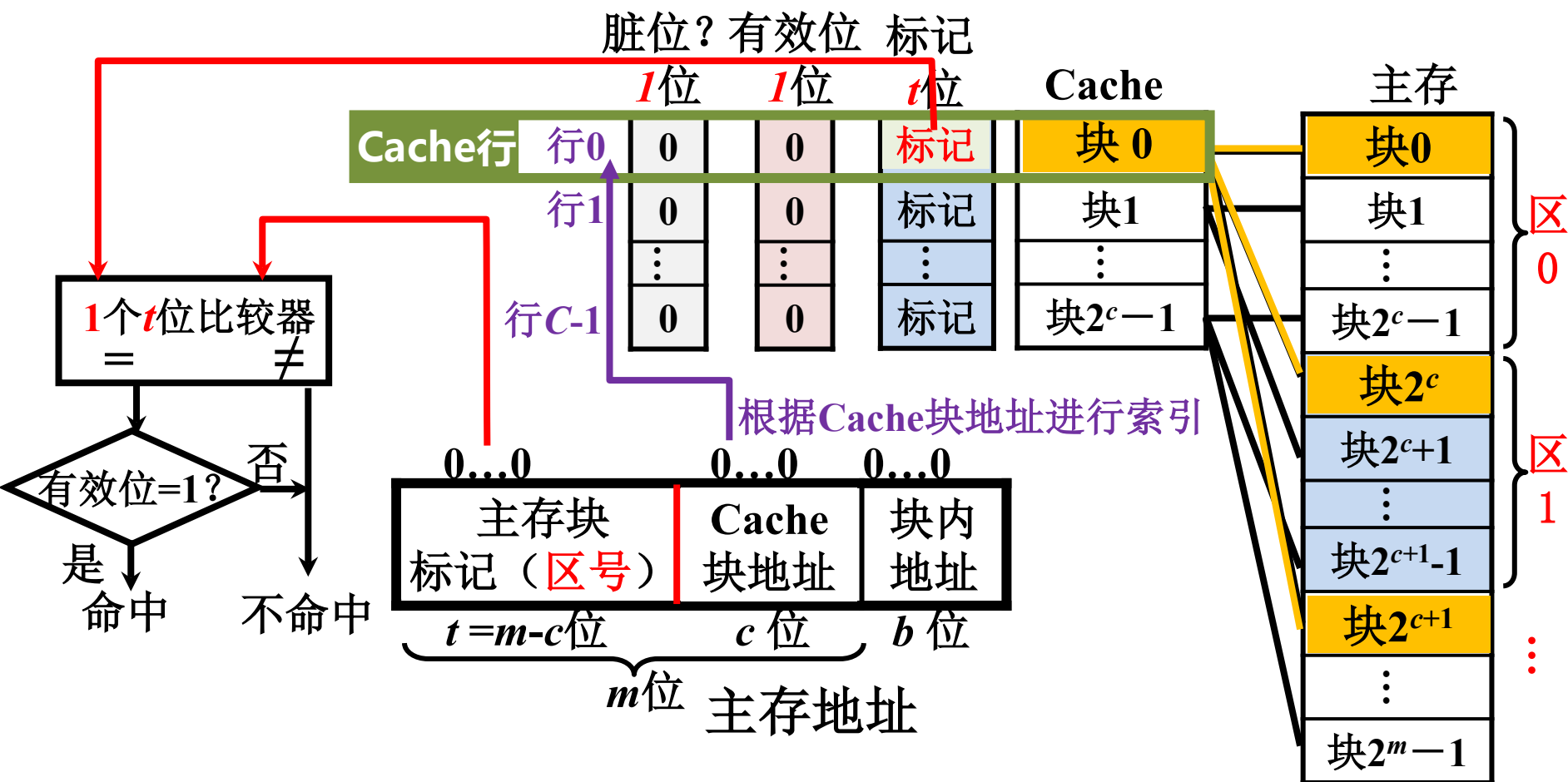
假设：主存块数 $M=2^m$ 块，Cache块数 $C=2^c$ 块，块长 $B=2^b$

每个主存块仅对应一个缓存块，主存块 j 映射到缓存块 i ，则有：

$$i = j \bmod C$$

1. 直接映射 有效位=1表示Cache块中数据有效。

假设CPU发送主存地址 $0\dots 0$ ，将地址剥离为3段： t 位0、 c 位0、 b 位0。

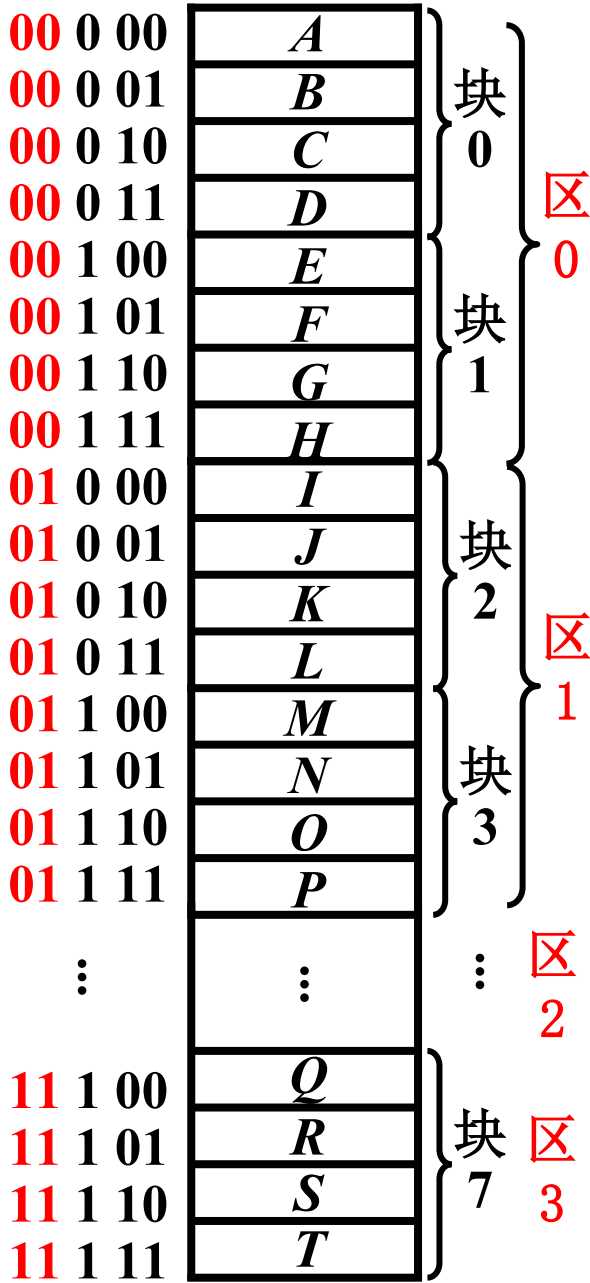
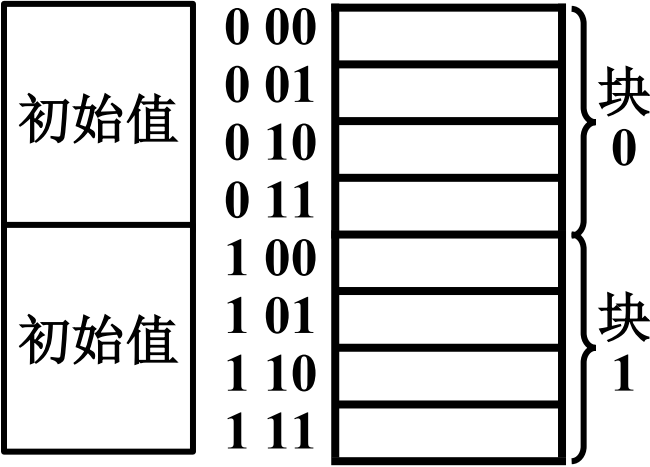


1. 直接映射

2位 标记

Cache

主存



块长 $B = 4$

主存地址:

主存 块标记	Cache 块地址	块内 地址
$t=2$ 位	$c=1$ 位	$b=2$ 位



9. 缓存的地址映射中，若每个主存块只能固定地映射到某个缓存块，称作（ ）。 **A**

A

直接映射

B

全相联映射

C

组相联映射

D

以上答案都不对

提交



10. 某计算机的Cache共有16块，采用直接映射方式。每个主存块大小为32字节，按字节编址。主存129号单元所在主存块应装入到的Cache块号是()。 (从0开始编号)

- A 0
- B 2
- C 4**
- D 6

提交





1. 直接映射

- 主存地址格式：

主存块标记	Cache块地址	块内地址
t 位	c 位	b 位
- 优点：主存块映射到Cache唯一固定块，速度快。不需要淘汰算法。
 - 利用Cache块地址进行索引，直接对比相应标记位即可；
 - 仅需 1 个 $t = m - c$ 位比较器。
- 缺点：灵活性差、利用率低，命中率低，适合大容量Cache。
- Cache容量 = 行大小 $\times$ 行数
= (脏位? + 有效位Valid + 标记位 + 数据块副本容量) $\times$ 行数

11. 【2015 直接映射、写回法】假定主存地址为32位，**按字节编址**。主存和cache之间采用直接映射方式，主存块大小为4个字，每字32位，采用写回法，则能存放**4K字**数据的cache的总容量至少是（ ） **位**。

- ☐ A 146K
- ☐ B 147K
- ☒ C 148K
- ☐ D 158K

提交





【例4.8】假设主存容量为512KB，Cache容量为4KB，每个字块为16个字，每个字32位。（未说明的情况下，默认为按字节编址）

- （1）Cache地址有多少位？可容纳多少块？
- （2）主存地址有多少位？可容纳多少块？
- （3）在直接映射方式下，主存的第几块映射到Cache中的第5块（设起始字块为第1块）？
- （4）画出直接映射方式下主存地址字段中各段的位数



【例4.8】 假设主存容量为**512KB**，**Cache**容量为**4KB**，每个字块为**16**个字，每个字**32**位。

(1) Cache地址有多少位？可容纳多少块？

解：(1) **Cache**的容量为**4KB=2¹²B**，则**Cache**的地址有**12**位。

又知每个字是**32**位=**4B**，每个字块**16**个字，

则每个字块的容量是**16×4B=64B**

则**Cache**中共有**4KB/64B=64**（块）



【例4.8】 假设主存容量为**512KB**，**Cache**容量为**4KB**，每个字块为**16**个字，每个字**32**位。

(2) 主存地址有多少位？可容纳多少块？

解：(2) 主存的容量为**512KB=2¹⁹B**，则主存的地址有**19**位。

又知每个字是**32**位=**4B**，每个字块**16**个字，

则每个字块的容量是**16×4B=64B**

则主存中共有**512KB/64B=8192**（块）



【例4.8】 假设主存容量为**512KB**，**Cache**容量为**4KB**，每个字块为**16**个字，每个字**32**位。

(3) 在直接映射方式下，主存的第几块映射到**Cache**中的第**5**块（设起始字块为第**1**块）？

解：(3) **Cache**共有**64**块，主存共有**8192**块，直接映射方式下，映射关系为： $i=j \bmod C$ ，其中为*i*缓存块号**5**，*j*为主存块号，**C**为缓存块数**64**。且起始字块为第**1**块。

$\therefore j$ 为**5**，**64+5**， $2 \times 64+5$ ，.....，

$(8192/64-1) \times 64+5=2^{13}-64+5$



【例4.8】假设主存容量为512KB，Cache容量为4KB，每个字块为16个字，每个字32位。

(4) 画出直接映射方式下主存地址字段中各段的位数

解：(4) 各段分别如图所示；

每个字块为 $16 \times 32 = 64\text{B}$ ，则字块内地址为 $b=6$ 位($2^6=64$)

缓存共有64块，则缓存字块地址同样为 $c=6$ 位。

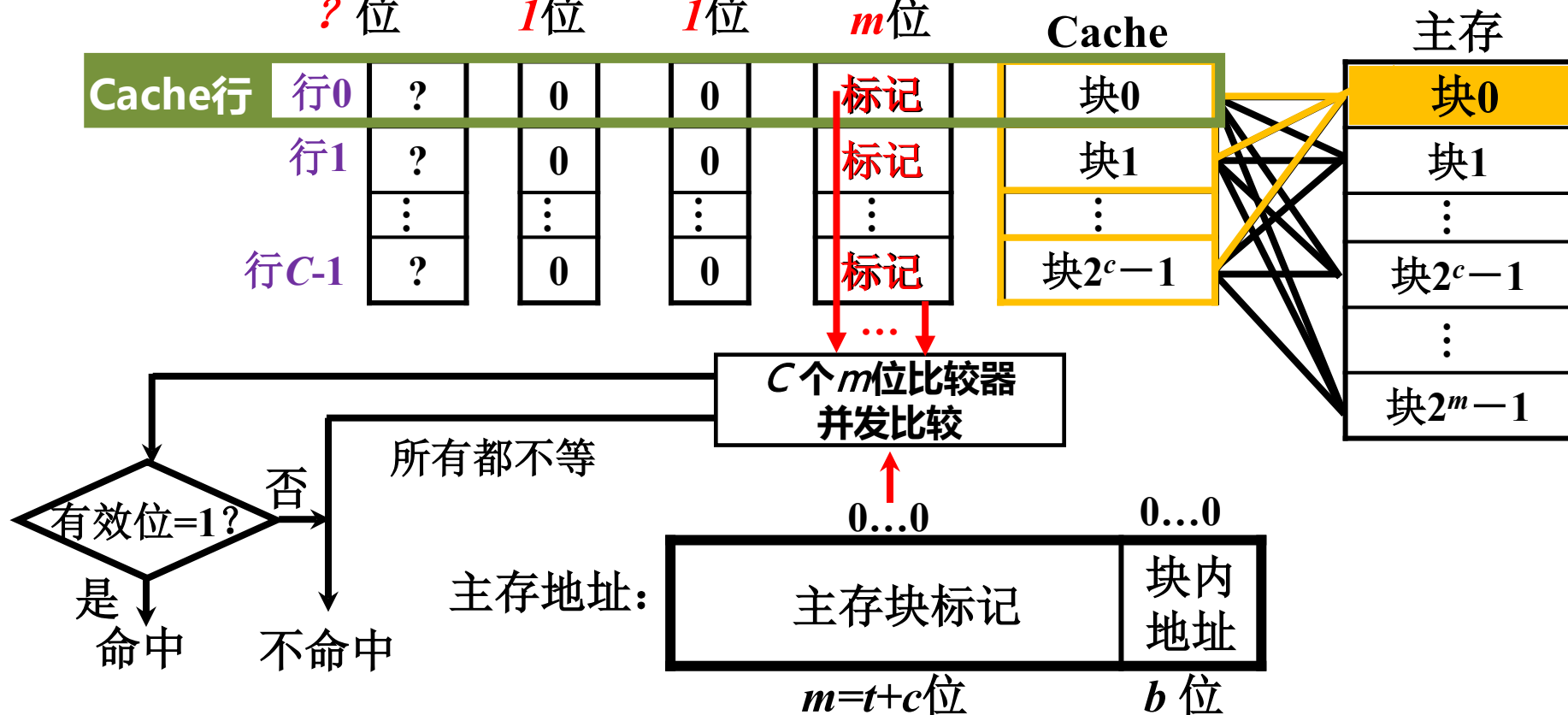
主存地址共有19位，则主存字块标记为 $t=19-6-6=7$ 位

主存字块标记	Cache字块地址	字块内地址
$t=7$ 位	$c=6$ 位	$b=6$ 位

假设CPU发送主存地址00...0，将地址剥离为2段： m 位0、 b 位0。

2. 全相联映射

替换算法 $?位$ 脏位? $1位$ 有效位 $1位$ 标记 $m位$



主存中的任一块可以映射到缓存中的任一块



12. 缓存的地址映射中，若每个主存块可以映射到任何一个缓存块，称作（ ）。 **B**

A

直接映射

B

全相联映射

C

组相联映射

D

以上答案都不对

提交





2. 全相联映射

- 主存地址格式：

主存块标记	块内地址
m 位	b 位
- 优点：主存块可映射到Cache任意一块，映射灵活，命中率高。
 - 利用Cache块地址进行索引，需要对比所有行标记位；
 - 需 C 个 m 位比较器。
- 缺点：速度慢，需要淘汰算法，结构复杂。一般较少使用。
- Cache容量 = 行大小 $\times$ 行数
= (淘汰算法位 + 脏位? + 有效位Valid + 标记位 + 数据块副本容量) $\times$ 行数



13. 全相联映射的灵活性最[高 1], 成本最 高 2]。
(填写高或低)



【例4.9】 假设主存容量为 $512\text{K} \times 16$ 位，Cache容量为 4096×16 位，块长为4个16位的字，访存地址为字地址。

(1) 在直接映射方式下，设计主存地址格式。

(2) 在全相联映射方式下，设计主存地址格式。



【例4.9】假设主存容量为 $512\text{K} \times 16$ 位，Cache容量为 4096×16 位，块长为4个16位的字，访存地址为字地址。

(1) 在直接映射方式下，设计主存地址格式。

解：

(1) 主存存储单元个数为 $512\text{K} = 2^{19}$ ，则主存地址共19位

块长 $B = 4 = 2^b$ ，则 $b = 2$ 位， $m = 19 - b = 19 - 2 = 17$ 位

Cache中的块数为 $4096 / 4 = 1024 = 2^c$ ，则 $c = 10$ 位；

主存字块标记 $t = m - c = 17 - 10 = 7$ 位

直接映射方式下，主存地址格式为：

主存字块标记	Cache字块地址	字块内地址
$t=7$ 位	$c=10$ 位	$b=2$ 位



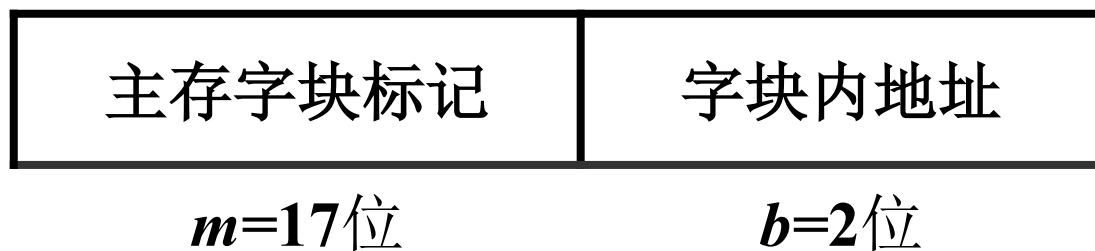
【例4.9】假设主存容量为 $512\text{K} \times 16$ 位，Cache容量为 4096×16 位，块长为4个16位的字，访存地址为字地址。

(2) 在全相联映射方式下，设计主存地址格式。

解：(2) 块长 $B=4=2^b$ ，则 $b=2$ 位

全相联映射方式下，主存字块标记 $m=19-b=19-2=17$ 位

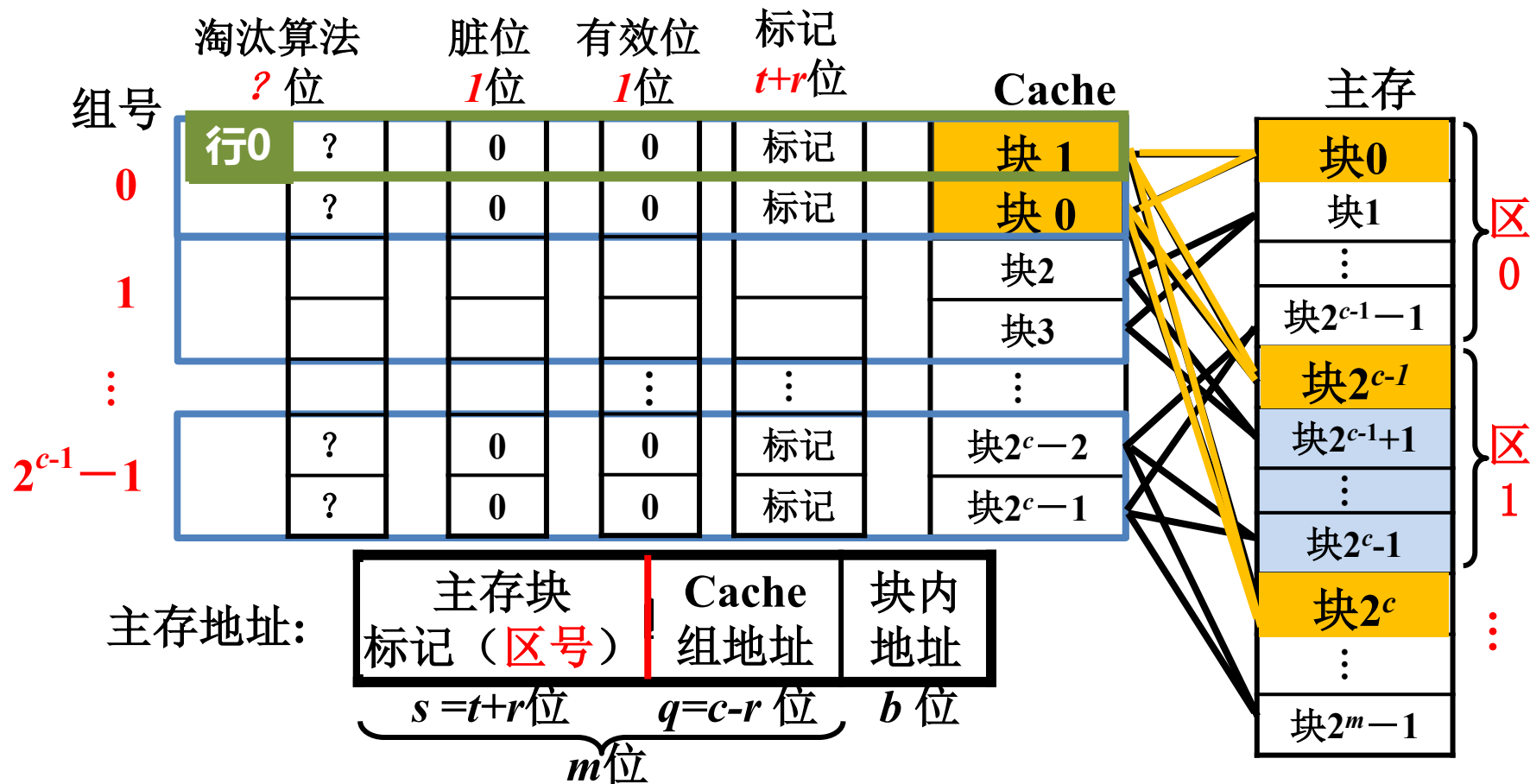
全相联映射方式下，主存地址格式为：



R 路组相联：Cache每组内 **$R=2^r$** 块，共 **$Q=2^q$** 组。则有 **$Q=C/R=2^{c-r}$**

3. 组相联映射

以2路组相联 ($r=1$) 为例

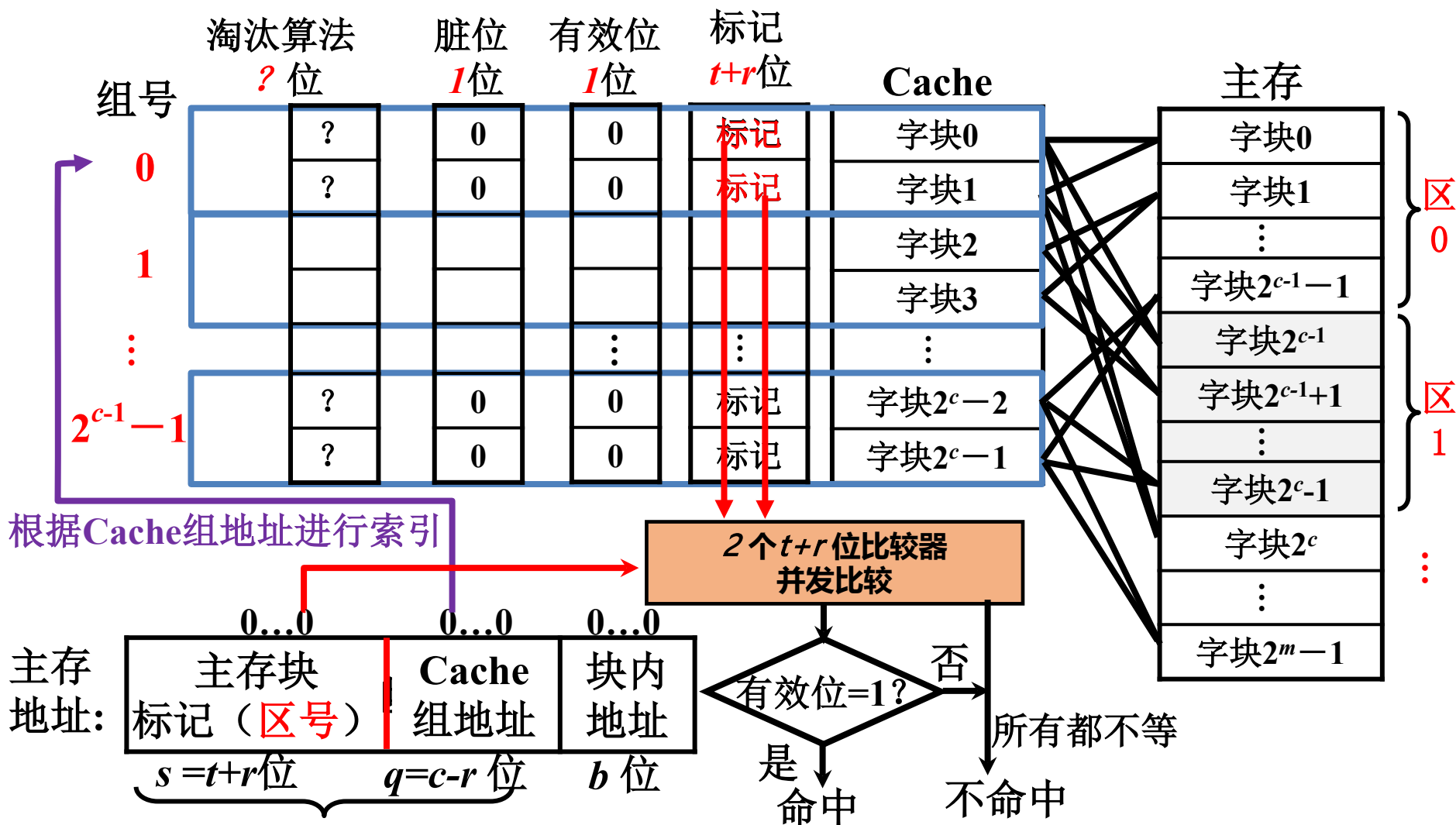


某一主存块 j 按模 Q 映射到缓存组 i 中的任一块，映射关系为：

组间直接映射 $i = j \bmod Q$ **组内全相联映射**

假设CPU发送主存地址00...0，将地址剥离为3段： **s 位0**、 **q 位0**、 **b 位0**。

3. 组相联映射 以2路组相联 ($r=1$) 为例





3. 组相联映射

- 主存地址格式：

主存块标记	Cache组地址	块内地址
-------	----------	------

$s=t+r$ 位 $q=c-r$ 位 b 位
- 特点：性能和复杂性都介于直接映射和全相联映射之间。
 - 利用Cache组地址进行索引，需要对比对应组内所有行标记位；
 - 需 R 个 $s = t+r$ 位比较器；
 - 当 $r=0$ 时，相当于直接映射；当 $r=c$ 时，相当于全相联映射。
- Cache容量 = 行大小 × 行数
= (淘汰算法位 + 脏位? + 有效位Valid + 标记位 + 数据块副本容量) × 行数
- 小容量Cache可采用全相联映射或组相联映射，大容量可采用直接映射方式。

14. Cache-主存的地址映射方法中最不灵活的是 **直接** 映射。

4路组相联映射是指每个缓存组中有 **4** 个缓存块。
(第2个空填写阿拉伯数字)

15. 【2022 组相联映射比较器个数和位数】若计算机主存地址为32位，按字节编址，某cache的数据区容量为32KB，主存块大小为64B，采用8路组相联映射方式，该cache中比较器的个数和位数分别为（ ）。

- ☒ A 8, 20
- ☐ B 8, 23
- ☐ C 64, 20
- ☐ D 64, 23

提交



16. 【2026 组相联映射地址】某计算机按字节编址，数据 Cache 共有 1024 行，采用 4 路组相联映射试，主存块大小为32B，若访问主存地址为1028的4字节数据，则该数据所在主存块对应的组号为（ ）。

- ☐ A 4
- ☐ B 16
- ☒ C 32
- ☐ D 64

提交





【例4.9】假设主存容量为 $512\text{K} \times 16$ 位，Cache容量为 4096×16 位，块长为4个16位的字，访存地址为字地址。

(3) 在二路组相联映射方式下，设计主存地址格式。

(4) 若主存容量为 $512\text{K} \times 32$ 位，块长不变，在四路组相联映射方式下，设计主存地址格式。



【例4.9】假设主存容量为 $512\text{K} \times 16$ 位，Cache容量为 4096×16 位，块长为4个16位的字，访存地址为字地址。

(3) 在二路组相联映射方式下，设计主存地址格式。

解：(3) 主存存储单元个数为 $512\text{K} = 2^{19}$ ，则主存地址共19位

块长 $B = 4 = 2^b$ ，则 $b = 2$ 位， $m = 19 - b = 19 - 2 = 17$ 位

在二路组相联映射方式下，一组内有2块，则Cache的组数为 $4096 \div 4 \div 2 = 512 = 2^q$ ，则 $q = 9$ 位

主存字块标记 $s = m - q = 17 - 9 = 8$ 位

二路组相联映射方式下，主存地址格式为：

主存字块标记	组地址	字块内地址
$s = 8$ 位	$c = 9$ 位	$b = 2$ 位



【例4.9】假设主存容量为 $512\text{K} \times 16$ 位，Cache容量为 4096×16 位，块长为4个16位的字，访存地址为字地址。

(4) 若主存容量为 $512\text{K} \times 32$ 位，块长不变，在四路组相联映射方式下，设计主存地址格式。

解：

(4) 主存容量为 $512\text{K} \times 32$ 位，为双字长存储器。在块长不变，字长不变的情况下。主存块数是原来的2倍。即主存块数为 $512\text{K} \times 2 \div 4 = 2^{18}$ ，则 $m=18$ 位



【例4.9】假设主存容量为 $512\text{K} \times 16$ 位，Cache容量为 4096×16 位，块长为4个16位的字，访存地址为字地址。

(4) 若主存容量为 $512\text{K} \times 32$ 位，块长不变，在四路组相联映射方式下，设计主存地址格式。

解：

(4) 在四路组相联映射方式下，Cache中的组数为： $4096 / (4 \times 4) = 2^8$ ，则 $q=8$ 位。

主存字块标记 $s=18-8=10$ 位

主存地址格式为：

主存字块标记	组地址	字块内地址
$s=10$ 位	$q=8$ 位	$b=2$ 位



【例4.11】 设某机主存容量为16MB，Cache的容量为8KB，每字块有8个字，每字32位。设计一个四路组相联映射的Cache组织。

- (1) 画出主存地址字段中各字段的位数。
- (2) 设Cache初态为空，CPU依次从主存第0，1，2，…，99号单元连续读出100个字（主存一次读出一个字），并重复此次序10次，问命中率是多少？
- (3) 若Cache的速度是主存速度的5倍，试问有Cache和无Cache相比，速度提高多少倍？
- (4) 系统的效率为多少？



【例4.11】 设某机主存容量为16MB，Cache的容量为8KB，每字块有8个字，每字32位。设计一个四路组相联映射的Cache组织。

(1) 画出主存地址字段中各字段的位数。

解：(1) 主存容量为16MB，则字节地址个数为 $16M=2^{24}$ ，则主存地址共24位。

每字块8个字，每字32位，则字块容量为 $8 \times 32b = 32B$ ，则块内字节地址个数为 $32=2^5$ ，则 $b=5$ 位

$$m=24-b=24-5=19\text{位}$$

Cache容量为8KB，则有 $8KB/32B=2^8$ 块，可得 $c=8$ 位；



【例4.11】 设某机主存容量为16MB，Cache的容量为8KB，每字块有8个字，每字32位。设计一个四路组相联映射的Cache组织。

(1) 画出主存地址字段中各字段的位数。

解：

在四路组相联映射方式下，一组内有4块，则Cache的组数为 $2^8 \div 4 = 64 = 2^6 = 2^q$ ，则 $q=6$ 位

主存字块标记 $s=m-q=19-6=13$ 位

四路组相联映射方式下，主存地址格式为：

主存字块标记	组地址	字块内地址
$s=13$ 位	$q=6$ 位	$b=5$ 位



【例4.11】 设某机主存容量为16MB，Cache的容量为8KB，每字块有8个字，每字32位。设计一个四路组相联映射的Cache组织。

(2) 设Cache初态为空，CPU依次从主存第0，1，2，…，99号单元连续读出100个字（主存一次读出一个字），并重复此次序10次，问命中率是多少？

解：

(2) 主存和Cache以块为单位进行信息交换。所以CPU读第0号单元时，未命中，必须访问主存，同时将该单元所在的主存块（共有8个字）调入Cache中第0组中的任一块内。接着CPU读第1~7号单元时均命中。



【例4.11】 设某机主存容量为16MB，Cache的容量为8KB，每字块有8个字，每字32位。设计一个四路组相联映射的Cache组织。

(2) 设Cache初态为空，CPU依次从主存第0，1，2，…，99号单元连续读出100个字（主存一次读出一个字），并重复此次序10次，问命中率是多少？

解：(2) 100个字占用的块数为 $100 \div 8 = 12.5$ ，即占用13块。

则CPU在第一次连续读100个字时，共有13次未命中。而后的9次循环读100个字均命中。

$$\text{命中率} = (100 \times 10 - 13) \div (100 \times 10) = 0.987$$



【例4.11】 设某机主存容量为**16MB**，**Cache**的容量为**8KB**，每字块有**8**个字，每字**32**位。设计一个四路组相联映射的**Cache**组织。

(3) 若**Cache**的速度是主存速度的**5**倍，试问有**Cache**和无**Cache**相比，速度提高多少倍？

解：(3) 设**Cache**的存取周期为 t ，主存的存取周期为 $5t$ ，则系统的平均访问时间为：

$$t_a = ht_c + (1-h)t_m = 0.987t + (1-0.987) \times 5t = 1.052t$$

性能提高了 $5t \div 1.052t - 1 = 3.75$ 倍



【例4.11】 设某机主存容量为**16MB**，**Cache**的容量为**8KB**，每字块有**8**个字，每字**32**位。设计一个四路组相联映射的**Cache**组织。

(4) 系统的效率为多少？

解： 系统的效率为

$$\begin{aligned} e &= \frac{\text{访问 Cache 的时间}}{\text{平均访问时间}} \times 100\% \\ &= \frac{t}{1.052t} \times 100\% \\ &= 95\% \end{aligned}$$



Cache—主存地址映射小结

□ 直接相联：某一主存块只能固定映射到 **唯一** 缓存块

不灵活

主存块标记

t 位

Cache块地址

c 位

块内地址

b 位

□ 全相联映射：某一主存块能映射到 **任一** 缓存块

成本高

主存块标记

m 位

块内地址

b 位

□ 组相联：某一主存块只能映射到 **唯一** 缓存组 中的 **任一块**

主存块标记

$s=t+r$ 位

Cache组地址

$q=c-r$ 位

块内地址

b 位



4.3.3 替换策略

1. 先进先出（FIFO）算法
2. 近期最少使用（LRU）算法
3. 随机算法



1. 先进先出 (FIFO) 算法

□ 原理：选择最早载入的Cache行进行替换。

替换指针指向最早行，可用一个计数器实现

【例】假设Cache初始状态为空，采用4路组相联映射，主存块{0, 1, 2, 3, 4}映射到同一Cache组。CPU访问12个主存块序列为：3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4。分析FIFO替换算法的访问过程，并计算命中率。

2/12

4
路
组
相
联
映
射

主存访问序列		3	2	1	0	3	2	4	3	2	1	0	4
Cache组内 块号	0	→3	3	3	3			4	4	4	→4	0	0
	1		2	2	2			→2	3	3	3	→3	4
	2			1	1			1	→1	2	2	2	→2
	3				0			0	0	→0	1	1	1
命中情况						√	√						
被替换块								3	2	1	0	4	3

颠簸

【例】分析上例采用LRU替换算法的访问过程，并计算命中率。

2. 近期最少使用（LRU）算法

4/12

□ 原理：根据程序访问的局部性原理，选择近期最久未被访问的行进行替换。每个Cache行需设置一个计数器。计数规则可如下：

- 命中时，命中行计数器清零，其余行计数器加 1；
- 未命中且有空行时，新装入行计数器置 0，其他非空行加 1；
- 未命中且无空行时，替换计数值最大的行，新装入行计数器置 0，其余行全加 1。

4
路
组
相
联
映
射

主存访问序列		3	2	1	0	3	2	4	3	2	1	0	4
Cache组内 块号	0	3 ⁰	3 ¹	3 ²	3 ³	3 ⁰	3 ¹	3 ²	3 ⁰	3 ¹	3 ²	3 ³	4 ⁰
	1		2 ⁰	2 ¹	2 ²	2 ³	2 ⁰	2 ¹	2 ²	2 ⁰	2 ¹	2 ²	2 ³
	2			1 ⁰	1 ¹	1 ²	1 ³	4 ⁰	4 ¹	4 ²	4 ³	0 ⁰	0 ¹
	3				0 ⁰	0 ¹	0 ²	0 ³	0 ⁴	0 ⁵	1 ⁰	1 ¹	1 ²
命中情况						√	√		√	√			
被替换块								1			0	4	3

成本?

【例】分析上例采用LRU替换算法的访问过程，并计算命中率。

近期最少使用（LRU）算法（408，最低成本，自学）

□ R 路组相联映射的计数器位数= $\log_2 R$ ，计数器的变化规则：

- ① 命中时，命中行计数器清零，比命中行低的计数器加1，其余不变；
- ② 未命中且有空行时，新装入行的计数器置0，其他非空行全加1；
- ③ 未命中且无空行时，替换计数值最大的行，新装入行计数器置0，其余行全加1。

		→ t													
4路组相联映射	主存访问序列		3	2	1	0	3	2	4	3	2	1	0	4	4/12
	Cache组内	0	3 ⁰	3 ¹	3 ²	3 ³	3 ⁰	3 ¹	3 ²	3 ⁰	3 ¹	3 ²	3 ³	4 ⁰	
	块号	1		2 ⁰	2 ¹	2 ²	2 ³	2 ⁰	2 ¹	2 ²	2 ⁰	2 ¹	2 ²	2 ³	
		2			1 ⁰	1 ¹	1 ²	1 ³	4 ⁰	4 ¹	4 ²	4 ³	0 ⁰	0 ¹	
		3				0 ⁰	0 ¹	0 ²	0 ³	0 ³	0 ³	1 ⁰	1 ¹	1 ²	
	命中情况					√	√			√	√				
被替换块									1			0	4	3	



3. 随机算法

□ **原理：** 随机确定替换的Cache行。这种策略的硬件最容易实现，而且速度也比前几种快；缺点是随机换出的数据很可能马上又用到，从而会降低命中率和工作效率。但这个缺点随着Cache容量的增大会减少。

模拟研究表明，随机替换算法的效率略逊于LRU算法，因此在虚拟存储器的TLB（快表）中为了提高替换速度，常采用随机替换算法。